

ERP SECURITY GATEWAY

End-to-End Production Implementation Playbook

A phased engineering plan for protecting the existing PHP/SQL ERP and Android client without waiting for historical labelled security logs.

Goal	Build a production-grade security layer in front of the existing ERP, starting with deterministic controls and telemetry, then adding adaptive anomaly detection.
Primary path	Android / Web → Edge/WAF → Reverse Proxy / API Gateway → Security Gateway → ERP Backend → SQL Database
Core principle	Do not make ML the firewall. Use deterministic controls for enforcement and ML for behavioral intelligence.
Dashboard	Simple and operational: traffic, blocked requests, suspicious events, top reasons, and incident details.

Status: Implementation baseline. Replace assumptions with verified ERP architecture, actual traffic/capacity measurements, and organizational requirements before production.

1. Executive Decision

Stop designing indefinitely and start with a narrow working security gateway. The first deliverable is a controlled path where every ERP API request is observed, evaluated, logged, and either allowed, limited, challenged, or blocked before reaching the existing ERP backend.

The absence of historical labelled traffic is not a blocker. The new gateway becomes the telemetry producer. Cold-start protection should use API contracts, authentication/authorization, rate limits, WAF signatures, threat intelligence, and conservative rules. Behavioral learning comes later, after telemetry quality and baseline stability are demonstrated.

The earlier architecture work identified a layered path involving edge protection, firewalling, reverse proxy/API gateway, a composite security gateway, ERP backend, database, and parallel analytics/SIEM components. This playbook keeps that direction but deliberately phases it to avoid unnecessary bespoke security machinery. `filecite{turn0file0}`

Non-negotiable principles

- Enforcement: the gateway must prevent unsafe traffic; logging alone is not protection.
- Fail safely: define behavior per endpoint and dependency; no blanket fail-open/fail-closed rule.
- Least privilege: every service and database identity gets only required access.
- Data minimization: security telemetry must not become a second sensitive-data repository.
- Explainability: every block/escalation has machine-readable reasons and evidence.
- Safe learning: suspicious or unreviewed traffic never automatically becomes normal training data.
- Production realism: availability, performance, rollback, observability, and operations are first-class.

2. What We Are Building — and Why

Problem

The ERP exposes valuable application functionality through web and mobile clients. A firewall controls network paths and a WAF catches many web attacks, but neither alone understands the complete application context or user behavior. The security layer therefore needs both enforcement and application-aware intelligence.

Solution

Introduce an independent security gateway in front of the ERP APIs. It becomes the controlled entry point for ERP traffic and provides policy enforcement, API validation, rate limiting, security detection, risk scoring, audit events, and a foundation for behavioral analytics.

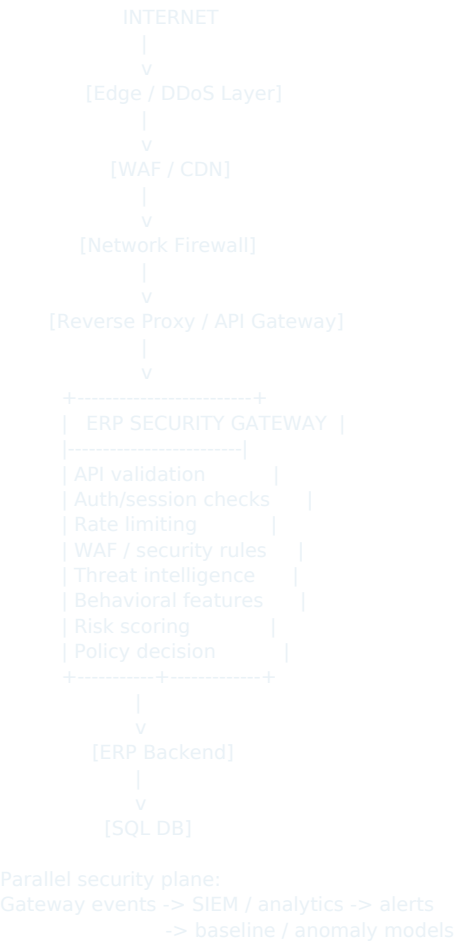


Why not put the whole security system inside Android?

- The client is not a trusted enforcement point; a compromised client can be modified or bypassed.
- Authoritative authentication, authorization, rate limiting, and blocking must remain server-side.
- Android should provide secure transport, secure token/key handling, and integrity signals where justified.
- Do not attempt to inspect all phone traffic; scope the system to ERP traffic.

3. Target Architecture

Reference deployment



Component responsibilities

Component	Responsibility	Avoid
Edge / DDoS	Volumetric protection and availability	Business logic
WAF	Known web attack patterns and protocol filtering	Behavioral identity decisions
API Gateway	Routing, TLS, quotas, API exposure	Complex ML decisions
Security Gateway	ERP-specific policy, risk, evidence, enforcement	Duplicating every security product
ERP Backend	Business logic and authoritative authorization	Trusting client roles
Database	Least-privilege persistence and auditing	Direct client exposure
SIEM	Search, correlation, alerting, investigation	Serving request path
ML	Behavioral intelligence	Being sole security authority

4. Phase 0 — Establish Ground Truth

Before implementation, document the real ERP. Do not assume database engine, Android stack, cloud provider, traffic volume, or API design.

Inventory

- PHP/runtime/web-server versions and deployment topology.
- Database engine/version and security-relevant schemas.
- API endpoints, methods, request/response schemas, authentication and token/session lifecycle.
- Android language/network stack, supported OS versions, secure storage, signing/release process.
- Existing firewall, load balancer, WAF, CDN, DNS, TLS, and certificates.
- Peak requests/second, concurrency, payload sizes, latency and uptime requirements.
- Existing application, auth, infrastructure, and database logs.
- Sensitive-data categories, retention requirements, backups and incident-response procedures.

Exit criterion

A verified system inventory and versioned API inventory exist. Unknowns are explicitly marked rather than silently assumed.

5. Phase 1 — Build the Gateway MVP

The first milestone is one complete vertical slice: request in → validate → score → decide → log → safe request forwarded to a test ERP.

Initial stack

- Python + FastAPI: security/risk service.
- Nginx: reverse proxy and routing.
- Redis: rate-limit and short-lived state.
- PostgreSQL or the actual DB later: security-event persistence.
- Docker Compose: reproducible development environment.
- Pytest: unit, integration and security regression tests.

```
erp-security/  
  gateway/  
    app.py config.py middleware/ policies/  
  detection/ risk/ audit/  
  backend/fake_erp/  
  tests/{unit,integration,security}/  
  deploy/{nginx,docker}/  
  docs/  
  .env.example  
  docker-compose.yml  
  README.md
```

MVP request lifecycle

1. Receive request
2. Generate correlation/request ID
3. Normalize safe metadata
4. Validate method/path/content type/schema
5. Check authentication context
6. Apply rate/abuse controls
7. Evaluate deterministic security rules
8. Calculate risk + reasons
9. Apply policy
10. Proxy allowed request
11. Record outcome + latency
12. Emit high-severity event if required

Initial decisions

Decision	Meaning	Action
ALLOW	Low-risk and policy-compliant	Forward

Decision	Meaning	Action
LIMIT	Likely abuse but insufficient evidence for block	Throttle/restrict
CHALLENGE	Higher-risk authenticated activity	Step-up/re-auth
BLOCK	Strong malicious/policy evidence	Do not forward
ALERT	Requires operator attention	Notify + retain evidence

6. Phase 2 — Deterministic Security Controls

This is the security foundation. ML comes later.

- API allowlisting: unknown paths/methods do not automatically reach the ERP.
- Schema validation: enforce fields, types, lengths, content types and limits.
- Authentication: reject missing/invalid credentials server-side.
- Authorization context: backend remains authoritative for object/resource permissions.
- Rate limiting: apply appropriate per-IP, account, session/token, installation, endpoint and global controls.
- Credential-abuse detection: repeated failures, distributed attempts and impossible velocity.
- WAF rules: injection, XSS, traversal, malformed HTTP and protocol abuse.
- Payload limits: request size, parameter count, nesting, upload limits and timeouts.
- Replay protection: for sensitive operations that require freshness.
- Egress controls: monitor and restrict unexpected outbound connections.

IDOR/BOLA warning

Object-level authorization depends on business context. The ERP backend must verify that the authenticated principal is allowed to access the requested resource. The gateway can add context and detect patterns but should not replace business authorization.

7. Phase 3 — Telemetry Without Historical Labels

The gateway creates the dataset. Historical manual labels are useful later, but not a prerequisite for MVP protection.

```
{
  "event_id": "uuid",
  "timestamp": "UTC",
  "request_id": "uuid",
  "principal_ref": "pseudonymous",
  "installation_ref": "pseudonymous",
  "method": "GET",
  "route": "/api/attendance",
  "body_size": 812,
  "status": 200,
  "decision": "ALLOW",
  "risk_score": 4,
  "reasons": [],
  "gateway_ms": 3,
  "upstream_ms": 41
}
```

Never log by default

- Passwords, raw access/refresh tokens, cookies, API keys or private keys.
- Full request/response bodies containing personal or business-sensitive data.
- Unnecessary stable device identifiers.
- Secrets embedded in exception messages or stack traces.

Baseline strategy

```
Observe + block obvious bad traffic
|
Measure low-risk allowed behavior
|
Analyst reviews baseline candidates
|
Freeze approved baseline version
|
```

Detect deviations

|

Review drift and update deliberately

Anti-poisoning rule

Allowed does not automatically mean normal. Suspicious sessions, high-risk activity and unreviewed anomalies must be excluded or quarantined from baseline/training data.

8. Phase 4 — Risk Engine

Use transparent scoring first. A score is a policy input, not an exact probability of attack.

```
risk = base
+ rule_severity
+ auth_anomaly
+ rate_abuse
+ endpoint_anomaly
+ reputation_signal
+ sequence_anomaly
- trusted_context
```

```
policy:
low    -> ALLOW
medium -> LIMIT / MONITOR
high   -> CHALLENGE / ALERT
critical -> BLOCK + ALERT
```

Do not hard-code arbitrary production thresholds. Tune against real traffic, capacity, false positives, false negatives and business impact.

9. Phase 5 — Behavioral Analytics and ML

Introduce ML only after telemetry and the baseline are trustworthy. The first question should be narrow: is this request/session behavior unusual compared with an approved baseline?

Useful features

- Requests per minute and burstiness.
- Unique endpoints accessed within a time window.
- Failed-authentication ratio.
- Endpoint transition/sequence patterns.
- Typical request size and parameter counts.
- Time-of-day behavior where meaningful.
- Account/installation relationship changes.
- Frequency of privileged operations.
- Response-code/error-burst patterns.

Model progression

Stage	Approach	Purpose
1	Rules + statistics	Explainable baseline
2	Isolation Forest or similar	Unlabelled anomaly detection
3	Temporal/sequence model if justified	Multi-step deviations
4	Supervised model once labels mature	Confirmed attack classification
5	Calibrated ensemble	Combine evidence

ML safety

- ML cannot silently change production policy.
- Version datasets, features, models and thresholds.
- Separate training, validation and test data.
- Monitor data and concept drift.
- Evaluate precision, recall, false positives/negatives, latency and operational impact.
- Maintain analyst review for ambiguous cases.
- Compare ML against the rules/statistics baseline; remove models that do not add measurable value.

10. Phase 6 — Android Security

The Android app is part of the trust model, but not the ultimate enforcement point.

- Use HTTPS/TLS correctly and avoid custom cryptography.
- Store sensitive credentials/tokens using Android secure mechanisms.
- Use short-lived credentials and secure refresh handling where appropriate.
- Consider Play Integrity or equivalent integrity signals where justified.
- Use app-generated installation identifiers instead of restricted hardware identifiers.
- For sensitive operations, use server-verifiable freshness/replay protection where needed.
- Use certificate pinning only with a robust rotation and recovery plan.
- Treat obfuscation as a reverse-engineering barrier, not secret management.

- Treat root/integrity signals as risk inputs rather than absolute proof of compromise.
- The server remains authoritative.

HMAC warning

A symmetric key embedded in an APK is not a durable secret. If request signing is required, use a realistic app/device-bound key strategy, secure key storage, server verification and replay protection. Never present a hardcoded client HMAC key as strong protection.

11. Phase 7 — Database and Backend Hardening

The gateway cannot compensate for insecure application code.

Backend

- Parameterized queries/prepared statements everywhere.
- Server-side authorization for every sensitive object/action.
- Modern adaptive password hashing.
- Secure session/token lifecycle and revocation.
- CSRF protection for browser cookie-based flows.
- Context-aware output encoding and XSS defenses.
- Strict file-upload validation and safe storage.
- Dependency/runtime patching.
- Safe error handling without stack traces or secrets.
- Production secrets separated from source code.

Database

- Only required application/security components can reach the database.
- Least-privilege DB roles.
- No direct database access from Android/web clients.
- Encryption in transit and protected backups.
- Audit privileged operations.
- Use a secrets manager where available.
- Define backup, restore, retention and recovery objectives.

Database proxy?

Use one only if justified by connection control, policy enforcement or isolation requirements. It is not a substitute for least privilege, segmentation, secret management and secure application code.

12. Phase 8 — Egress Protection

A compromised ERP server can become an exfiltration or command-and-control source. Design outbound visibility from the beginning.

ERP servers

```
|
+--> approved integrations
+--> approved DNS
+--> approved external services
|
X--> unexpected destinations
X--> suspicious DNS
X--> unusual protocols
X--> abnormal outbound volume
```

Initial controls

- Outbound allowlist where feasible.
- Central DNS visibility.
- Monitor destination/domain changes.
- Alert on unexpected protocols and unusual volume.
- Separate server roles to limit outbound blast radius.

- Retain investigation evidence without unnecessary payload logging.

13. Phase 9 — Simple, High-Value Dashboard

The dashboard is an operations console, not a data-science project.

ERP SECURITY			
Requests	Blocked	Suspicious	Alerts
1.28M	4,821	932	17
Top security reasons			
SQLi / WAF	1,201		
Auth abuse	824		
Rate abuse	511		
Unknown API	203		
Recent high-severity events			
10:32 HIGH	Repeated login failures		
10:34 HIGH	Malicious request pattern		
10:41 MED	Abnormal API sequence		
[Open incident] [View events] [Policy status]			

Keep only five first-release views

- Overview — traffic, blocked/suspicious counts, alerts, gateway health.
- Events — searchable security events with reason, endpoint and decision.
- Incident — timeline/evidence for a serious event.
- Policies — enabled rules, versions and rollback state.
- Health — instances, latency, event pipeline and rule/model version.

14. Phase 10 — Security Agents

Introduce agents after detection and evidence are reliable. Agents should orchestrate investigation and approved operations, not become an uncontrolled autonomous security authority.

Agent	Purpose	Autonomy
Detection	Correlate rules/behavior/telemetry	Read/score
Investigation	Build timeline and evidence summary	Read-only
Threat Intel	Enrich indicators	Read/enrich
Response	Execute approved containment	Pre-approved/reversible only
Reporting	Generate summaries/metrics	Read-only

High-impact actions such as disabling accounts, changing firewall policy or taking infrastructure offline should require explicit authorization unless an approved emergency policy exists.

15. Production Reliability and Deployment

Security controls that cause outages become security problems. Every dependency needs a documented reliability contract.

Define for every dependency

- What happens if WAF, risk service, Redis or event pipeline is unavailable?
- Which endpoints can operate in degraded mode?
- Which sensitive operations must stop when verification is unavailable?
- How is degraded behavior logged?
- How is recovery tested?

Deployment

Development
-> unit tests
-> integration/security tests
-> staging
-> synthetic traffic
-> canary
-> observe
-> gradual production rollout
-> rollback if security/SLO criteria fail

Reliability requirements

- Multiple gateway instances where availability requires it.
- Bounded timeouts, queues and concurrency.
- Event pipelines must not block requests indefinitely.
- Health checks test useful dependencies.
- Policies and models are versioned and rollbackable.
- Measure current ERP p50/p95/p99 latency and set a gateway performance budget.

16. Testing Strategy

Prove both malicious traffic is controlled and legitimate ERP workflows remain functional.

Layer	Examples
Unit	Rules, scoring, normalization, redaction
Integration	Gateway→ERP, auth, Redis/event failures
Security	OWASP API risks, injection, malformed requests, abuse
Load	Expected/peak concurrency and bursts
Resilience	Dependency failures, restart, latency, event sink outage
Adversarial	Safe isolated evasion tests
Regression	Known-good and known-bad traffic corpus
Mobile	TLS, tokens, integrity, replay behavior

Key metrics

- False positives and false negatives by meaningful threat/workflow class.
- Detection latency.

- Gateway overhead and error rate.
- Event-loss rate.
- Model/rule rollback frequency.
- Mean time to detect/respond for serious incidents.

17. Phased Delivery Roadmap

Phase	Deliverable	Exit criterion
0	System/API inventory + threat model	Architecture reflects actual ERP
1	Gateway MVP + fake ERP	End-to-end path works
2	Rules, schema, rate limits	Known abuse controlled
3	Telemetry + event store	Reliable redacted events
4	Staging ERP integration	Legitimate workflows pass
5	Baseline/statistics	Deviations visible
6	ML anomaly model	Measurable value over baseline
7	Android hardening	Client/server contract verified
8	Egress monitoring	Outbound risk visible
9	Simple dashboard/incidents	Operators can act
10	Agents	Only approved actions automated
11	Production rollout	Canary + rollback + SLO evidence

Do not wait for the final phase to demonstrate progress. Phase 1 is already a meaningful security deliverable.

18. Master Prompt — Principal Security Architect

ROLE

Act as a Principal Security Architect, Staff Backend Engineer, Network Security Engineer, API Security Engineer, Android Security Engineer, SRE, and ML Security Engineer. Design a production ERP security layer. Be skeptical, evidence-driven, and implementation-oriented.

MISSION

Design/review a security gateway in front of an existing ERP. It must protect APIs, enforce policy, observe traffic, detect abuse, produce auditable evidence, and progressively add behavioral intelligence.

CONSTRAINTS

- Existing ERP backend must remain usable.
- Historical labelled logs may not exist.
- Do not assume DB engine, cloud provider, traffic volume, or Android stack.
- ML is not the sole enforcement mechanism.
- Minimize sensitive telemetry.
- Prefer mature components over unnecessary custom products.
- High-impact autonomous actions require explicit policy/approval.

REQUIRED OUTPUT

1. Assumptions and unknowns.
2. Request/data-flow architecture.
3. Trust boundaries and attack surfaces.
4. Minimum viable gateway.
5. Exact controls and enforcement locations.
6. Event schemas and redaction.
7. Cold-start detection without labels.
8. Baseline-learning and anti-poisoning controls.
9. ML features, models, metrics and rollback.
10. Android/server responsibilities.
11. Backend/database dependencies.
12. Egress controls.
13. HA, failure modes and degraded mode.
14. Testing and acceptance criteria.
15. Architectural weaknesses and alternatives.
16. Executable phased implementation plan.

CRITICAL REVIEW

- Separate facts, assumptions and recommendations.
- Never claim a control is absolute.
- Never recommend hardcoded mobile secrets.
- Anomaly is not proof of malicious intent.
- Never invent traffic or performance numbers.
- Never log passwords, tokens or unnecessary PII.
- Define fail behavior per dependency/endpoint.
- Challenge bespoke components duplicating mature products.
- Prioritize correctness, availability, observability and rollback.

FINAL FORMAT

Architecture -> Threat Model -> Controls -> Data -> Detection -> ML -> Mobile
-> Database -> Egress -> Reliability -> Testing -> Deployment -> Risks
-> Phased plan -> Acceptance criteria.

19. Master Prompt — Senior Implementation Engineer

Act as the senior engineer implementing the approved ERP Security Gateway. Do not produce a toy demo. Produce maintainable, testable, secure, production-oriented code while clearly separating prototype shortcuts from production requirements.

IMPLEMENTATION RULES

1. Start with the smallest end-to-end vertical slice.
2. Keep security decisions deterministic and explainable.
3. Use configuration, not hardcoded secrets.
4. Validate every untrusted input.
5. Enforce request size, timeout and concurrency limits.
6. Add structured logging with mandatory redaction.
7. Add correlation/request IDs.

8. Make security decisions auditable.
9. Use least-privilege service identities.
10. Add unit, integration, security and failure-mode tests.
11. Version policies and models.
12. Give every dependency a timeout and degraded-mode behavior.
13. Never expose stack traces, credentials or internal topology.
14. Never weaken security just to pass tests.
15. Backend authorization remains authoritative.

FOR EACH STEP OUTPUT

- Files to create/change
- Purpose
- Interfaces/contracts
- Configuration
- Tests
- Security implications
- Observability
- Failure behavior
- Rollback
- Definition of done

BEFORE CODING

List assumptions. Ask only questions that block implementation; otherwise choose a safe default and proceed.

AFTER CODING

Review the implementation for injection, authentication, authorization, SSRF, deserialization, traversal, secret exposure, logging, race conditions, DoS, dependency and configuration risks.

20. Master Prompt — Detection / ML Engineer

Act as a senior security detection engineer and ML security engineer.

OBJECTIVE

Build behavioral detection for ERP traffic without assuming historical labelled data exists.

RULES

- Deterministic controls come first.
- Anomaly != malicious.
- Never train on all allowed traffic automatically.
- Maintain candidate, approved, quarantined and rejected data states.
- Version datasets, features, models and thresholds.
- Separate training, validation and test data.
- Monitor drift.
- Prefer interpretable features.
- Compare ML against a rules/statistics baseline.
- Define false-positive and false-negative cost.
- Explain every high-risk score.

OUTPUT

1. Threat behaviors.
2. Feature definitions.
3. Data-quality requirements.
4. Baseline construction.
5. Model candidates and rationale.
6. Training/evaluation pipeline.
7. Metrics by threat class and legitimate workflow.
8. Threshold-selection method.
9. Drift detection.
10. Poisoning defenses.
11. Deployment/rollback.
12. Safe policy-engine integration.

DO NOT claim all attacks are detectable, use accuracy alone, automatically block solely on an unsupervised anomaly, or store raw credentials/unnecessary payloads.

21. Master Prompt — Authorized Security Validation

Act as an authorized ERP application security assessor.

SCOPE

Only test explicitly approved isolated/staging assets. Do not perform destructive actions or uncontrolled traffic generation.

TEST

Authentication, session handling, authorization/IDOR/BOLA, SQL injection, XSS, CSRF where applicable, SSRF, traversal, file upload, API schema bypass, rate-limit bypass, credential abuse, replay, token misuse, resource exhaustion, egress abuse, Android/server trust-boundary errors, logging integrity, and gateway-bypass attempts.

FOR EACH FINDING

severity -> affected component -> precondition -> safe reproduction summary
-> evidence -> business impact -> remediation -> regression test -> verification.

SUCCESS

Malicious traffic is controlled as designed; legitimate workflows remain functional; decisions have evidence; sensitive information is not leaked; the gateway cannot be trivially bypassed to reach the backend.

22. Master Prompt — Production Readiness Reviewer

Act as the final production-readiness reviewer for the ERP Security Gateway.

REVIEW

Authentication, authorization, input validation, WAF/API security, rate limits,

secrets, TLS, logging/redaction, DB permissions, segmentation, egress, HA, timeouts, circuit breakers, event loss, monitoring, alerting, backup/recovery, deployment/rollback, dependencies, Android assumptions, ML governance and agent permissions.

OUTPUT

- A. BLOCKERS — must fix
- B. HIGH RISK — fix before broad rollout
- C. MEDIUM — planned hardening
- D. LOW — improvement

For every blocker/high-risk item:

evidence -> risk -> exact remediation -> test -> owner -> acceptance criterion.

Do not approve production from documentation alone. Require evidence from tests, configuration, deployment checks and observed behavior.

23. Definition of Done — First Real Milestone

Do not move to ML until these are true.

- A request enters the gateway and reaches ERP only when policy allows it.
- Unknown paths/invalid methods are handled according to policy.
- Authentication failures and abusive rates are detected.
- Known injection/malformed-request classes are tested and controlled.
- Security events have correlation IDs and mandatory redaction.
- Gateway latency/error overhead is measured.
- Dependency failure behavior is documented and tested.
- Security decisions are explainable from stored evidence.
- Staging deployment can be rolled back.
- Known-good ERP workflows have automated regression tests.
- A basic operator can inspect blocked/suspicious events without a complex dashboard.
- Telemetry is sufficient to establish a reviewed baseline.

24. Immediate 7-Day Execution Plan

Day	Work
1	Inventory ERP/API/auth/database stack; create repository; define threat model and MVP boundary.
2	Build fake ERP + Nginx + gateway skeleton; establish request IDs.
3	Implement API allowlist/schema validation, rate limits and structured events.
4	Implement deterministic rules and transparent risk/policy engine.
5	Build unit/integration/security tests and dependency-failure tests.
6	Connect to staging ERP; verify workflows; tune policy; measure latency.
7	Run controlled security assessment; fix blockers; freeze MVP; begin baseline collection.

The first week is not about an AI model. It is about creating a security-controlled, observable and testable request path.

25. Final Engineering Position

Present this as a layered ERP security platform, not simply an AI firewall. The gateway is the enforcement point; WAF/API controls handle known and protocol-level attacks; the ERP handles business authorization; telemetry provides evidence; behavioral analytics finds deviations; ML adds intelligence after the baseline is trustworthy; agents automate investigation and only narrowly approved responses.

BUILD -> OBSERVE -> ENFORCE -> MEASURE -> BASELINE
-> DETECT -> VALIDATE -> AUTOMATE CAREFULLY -> HARDEN

Not:

BUILD AI -> hope it understands the ERP -> put it in front of production.

Start with the gateway vertical slice. Everything else can grow around it.

Implementation note: Replace assumptions with verified ERP architecture, actual traffic/capacity measurements, applicable policies, and current component security guidance before production deployment.